



UNIVERSITY OF MILAN

Computer Science
Statistical Methods for Machine Learning

SVM & Logistic Regression for Wine-Quality Prediction

Student:

Alessandro Sarchi

ID number: 66046A

alessandro.sarchi@studenti.unimi.it

Professor:

Nicolò Cesa-Bianchi

Computer Science Department

Academic Year 2024/2025

Contents

Abstract	1
Declaration of Authorship	1
1 Introduction	1
2 Dataset Analysis	2
2.1 Exploration	2
2.1.1 Initial exploration and data cleaning	2
2.1.2 Data preparation and train-test split	2
2.1.3 Distribution analysis	2
2.1.4 Correlations and multicollinearity	3
2.2 Preprocessing	3
2.2.1 Categorical variable encoding	3
2.2.2 Feature Engineering	3
2.2.3 Scaling	3
2.2.4 Data export	4
3 Models Implementation	4
3.1 Kernel Functions	4
3.2 Support Vector Machines	4
3.2.1 Linear SVM: PEGASOS with Parameter Averaging	5
3.2.2 Kernel SVM: RKHS Formulation	5
3.3 Logistic Regression	6
3.3.1 Linear Logistic Regression	6
3.3.2 Kernel Logistic Regression	7
4 Experimental Analysis	8
4.1 Hyperparameter Tuning Methodology	8
4.2 Linear Models	8
4.2.1 Hyperparameter Search Space	8
4.2.2 Grid Search Results	8
4.2.3 Learning Curve Analysis	9
4.2.4 Loss Convergence Analysis	10
4.2.5 Test Set Performance	11
4.3 Kernel Models	11
4.3.1 Hyperparameter Search Space	11
4.3.2 Grid Search Results	12
4.3.3 Kernel Comparison	12
4.3.4 Loss Convergence Analysis	13
4.3.5 Test Set Performance	13
4.4 Comparative Analysis	14

Abstract

This project addresses the binary classification of wine quality through from-scratch implementations of **Support Vector Machines (SVM)** and **Logistic Regression (LR)**. Using the UCI Wine Quality dataset (6,500 samples, 11 physicochemical features), we classify wines as *good* (score ≥ 6) or *poor* (score < 6) and compare linear versus kernel-based approaches. Both algorithms employ stochastic optimization with **parameter averaging**: SVM via PEGASOS with hinge loss, and LR via regularized gradient descent with sigmoid loss. For kernel methods, we implement the representer theorem formulation, maintaining support vectors with coefficient averaging to improve generalization. The experimental protocol combines 5-fold cross-validation for hyperparameter selection (exploring iteration counts, regularization strengths, and kernel parameters) with comprehensive evaluation on held-out test data. Results demonstrate that **kernel methods substantially outperform linear models**, with Gaussian kernels achieving the best results across both algorithms. The optimal configuration, **Gaussian SVM** with $\lambda = 0.001$ and $\gamma = 1.0$, attains **76.3% accuracy** and 81% F1-score, representing a 5.3 percentage point improvement over linear SVM (71.0%). Learning curve analysis reveals proper convergence by 2000 iterations, while loss tracking confirms effective optimization across all configurations. Although kernel methods exhibit larger train-test gaps (4-5%) compared to linear models (1.5-2%), their superior test performance validates the hypothesis that wine quality depends on complex, non-linear interactions among physicochemical properties. The implementation provides a complete framework for kernel-based classification with theoretical grounding in RKHS and empirical validation on real-world sensory data.

Declaration of Authorship

I declare that this material, which I now submit for assessment, is entirely my own work and has not been taken from the work of others, save and to the extent that such work has been cited and acknowledged within the text of my work. I understand that plagiarism, collusion, and copying are grave and serious offences in the university and accept the penalties that would be imposed should I engage in plagiarism, collusion or copying. This assignment, or any part of it, has not been previously submitted by me or any other person for assessment on this or any other course of study.

1 Introduction

This project addresses the problem of automatic wine quality prediction through machine learning techniques. The objective is to develop a binary classification system capable of distinguishing between *good* quality wines (score ≥ 6) and *poor* quality wines (score < 6) based exclusively on measurable physicochemical properties.

The study implements and compares two fundamental algorithms: **Support Vector Machine (SVM)** and **Logistic Regression (LR)**, both developed from scratch to ensure a thorough understanding of the underlying mathematical principles. The analysis uses the **Wine Quality dataset** from the UCI Machine Learning Repository¹, which includes approximately 6,500 samples of red and white wines characterized by 11 physicochemical properties (fixed acidity, volatile acidity, citric acid, residual sugars, chlorides, free and total sulfur dioxide, density, pH, sulfates, and alcohol content).

The project provides a comparative analysis of the performance of both methodological approaches, evaluating their effectiveness in wine quality classification and offering insights into the strengths and weaknesses of each algorithm in the specific context of enological analysis.

¹dataset available here: <https://archive.ics.uci.edu/dataset/186/wine+quality>

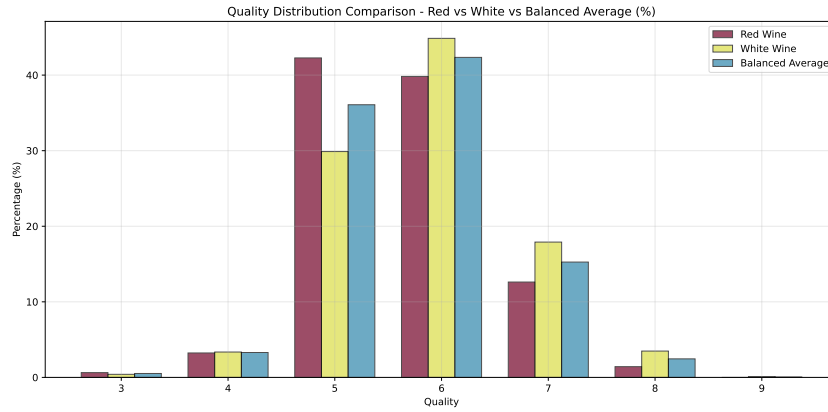


Figure 1: *Quality Distribution.*

2 Dataset Analysis

2.1 Exploration

The project began with a thorough **exploratory analysis of the dataset**, a fundamental phase for understanding the nature and characteristics of the data before proceeding with algorithm implementation.

2.1.1 Initial exploration and data cleaning

Initially, the two datasets (red wine and white wine) were analyzed separately, examining the number of samples and main descriptive statistics for each feature, including means, medians, and percentiles. This preliminary analysis allowed us to understand the variability domain of the different physicochemical properties. Subsequently, **dataset consistency** was verified by checking for null or missing values that would have required specific treatment strategies. Fortunately, the dataset proved to be complete and free of missing values.

2.1.2 Data preparation and train-test split

The two datasets were merged to simplify subsequent analysis. The `split_train_test()` function was then implemented, inspired by interface and nomenclature from **scikit-learn** to ensure familiarity and consistency in use. This design choice was maintained constant throughout the project to facilitate usage by those with experience with that library. The function accepts as parameters the examples X , labels y , a `random_state` seed to ensure reproducibility (set to default 42 throughout the project), and the `stratify` parameter to maintain balanced the target class distribution in the train and test sets. This separation was performed immediately **to avoid any form of data leakage** and treat test data as completely new during final evaluation.

2.1.3 Distribution analysis

Analysis of the target variable distribution revealed that most wines are concentrated on scores 5 (approximately 36%) and 6 (approximately 42%). Regarding binary classification (*good* vs *bad*), the dataset presents a **slight imbalance** with 60% of wines classified as *good* and 40% as *bad*. Examination of individual feature distributions showed that most follow approximately normal distributions, although with varying degrees of skewness and kurtosis. A **significant presence of outliers** beyond the $1.5 \cdot \text{IQR}$ (InterQuartile Range) threshold emerged for almost all features. The only exception is the *alcohol* feature, which presents a distribution that is difficult to interpret.

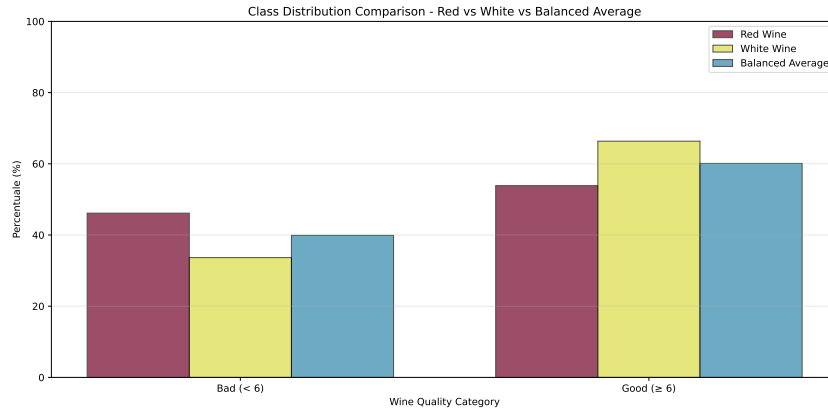


Figure 2: *Class Distribution.*

2.1.4 Correlations and multicollinearity

Correlation matrix analysis identified **potential multicollinearity problems** using a $|0.6|$ threshold to highlight significant correlations. Two pairs of strongly correlated features emerged:

- *free sulfur dioxide* - *total sulfur dioxide*: a logically predictable correlation given the chemical nature of the variables
- *density* - *alcohol*: a less intuitive correlation for those not expert in the wine sector

This analysis provided the foundation for considering feature selection strategies in subsequent phases of the project.

2.2 Preprocessing

2.2.1 Categorical variable encoding

The first preprocessing step involved **encoding categorical features**, replacing non-numerical variables with numerical values 0 and 1 to make them compatible with machine learning algorithms.

2.2.2 Feature Engineering

Subsequently, we addressed the management of problematic features identified in the correlation analysis:

- ***Sulfur dioxide treatment***: Given the high correlation between *free sulfur dioxide* and *total sulfur dioxide*, a new feature called *free sulfur dioxide ratio* was created that captures the ratio between these two variables, eliminating informational redundancy and providing a more compact representation of the relationship.
- **For the *density* - *alcohol* pair**, we chose to maintain both features since the correlation, while high, provides complementary information useful for predictive models. The same strategy was applied to the correlation that emerged after encoding between ***volatile acidity* and *wine type***, considering this relationship more as an advantage than a problem for the predictive capacity of the models.

2.2.3 Scaling

The **StandardScaler** class was implemented, following the same design philosophy adopted for other project components. This class operates through two main methods: `fit_transform()`:

used on training data (`X_train`) to calculate mean and standard deviation of each feature and simultaneously apply the transformation `transform()`: used on test data applying parameters (mean and standard deviation) calculated exclusively on training data, thus avoiding data leakage. Standardization applies the formula

$$X_{scaled} = \frac{X_{train} - \mu}{\sigma}$$

maintaining the original shape of distributions but centering each feature on mean $\mu = 0$ and standard deviation $\sigma = 1$. This process ensures that no feature has disproportionate weight due to its numerical scale.

2.2.4 Data export

At the end of preprocessing, the transformed **data was exported** to freeze the dataset state and facilitate import in subsequent notebooks dedicated to algorithm implementation and evaluation.

3 Models Implementation

Both SVM and Logistic Regression share a unified architecture that supports both linear and kernel-based classification. The implementation employs stochastic optimization with parameter averaging to enhance generalization performance. This section describes the mathematical foundations and algorithmic details of each component.

3.1 Kernel Functions

The kernel function serves as the fundamental building block for non-linear classification in both models. The implementation provides two kernel types that implicitly map input data to higher-dimensional feature spaces without explicit computation of the transformation $\phi(\mathbf{x})$.

Polynomial Kernel The polynomial kernel of degree d is defined as:

$$K_{poly}(\mathbf{x}_1, \mathbf{x}_2) = (1 + \mathbf{x}_1^\top \mathbf{x}_2)^d$$

This kernel corresponds to the inner product in a feature space containing all polynomial combinations up to degree d . The constant term 1 ensures that the kernel includes lower-degree terms, enabling the model to learn both simple and complex decision boundaries.

Gaussian Kernel The Gaussian kernel with bandwidth parameter γ is defined as:

$$K_{gauss}(\mathbf{x}_1, \mathbf{x}_2) = \exp\left(-\frac{1}{2\gamma}\|\mathbf{x}_1 - \mathbf{x}_2\|^2\right)$$

This kernel maps data into an infinite-dimensional feature space, providing maximum flexibility for capturing non-linear patterns. The parameter γ controls the influence radius of individual training examples: smaller values produce smoother decision boundaries, while larger values allow more localized, complex boundaries.

3.2 Support Vector Machines

The SVM implementation follows the PEGASOS algorithm [1], which solves the primal SVM optimization problem through stochastic gradient descent. The algorithm admits two distinct operational modes depending on the kernel type.

3.2.1 Linear SVM: PEGASOS with Parameter Averaging

For linear kernels, the model maintains an explicit weight vector $\mathbf{w} \in \mathbb{R}^n$ and bias term $b \in \mathbb{R}$. The decision function takes the form:

$$f(\mathbf{x}) = \text{sgn}(\mathbf{w}^\top \mathbf{x} + b)$$

Stochastic Gradient Descent At each iteration $t \in \{1, \dots, T\}$, the algorithm randomly samples an example $(\mathbf{x}_t, y_t)$ and computes the margin:

$$m_t = y_t(\mathbf{w}_t^\top \mathbf{x}_t + b_t)$$

The learning rate follows the schedule $\eta_t = \frac{1}{\lambda t}$, where λ is the regularization parameter. This time-dependent decay ensures convergence while maintaining sufficient plasticity in early iterations.

Hinge Loss and Weight Updates The update rule depends on whether the current example violates the margin condition $m_t < 1$:

$$\mathbf{w}_{t+1} = \begin{cases} \left(1 - \frac{1}{t}\right) \mathbf{w}_t + \eta_t y_t \mathbf{x}_t & \text{if } m_t < 1 \\ \left(1 - \frac{1}{t}\right) \mathbf{w}_t & \text{otherwise} \end{cases}$$

$$b_{t+1} = \begin{cases} b_t + \eta_t y_t & \text{if } m_t < 1 \\ b_t & \text{otherwise} \end{cases}$$

The factor $\left(1 - \frac{1}{t}\right)$ implements implicit regularization through weight decay, corresponding to the regularization term in the primal objective. When $m_t \geq 1$, the example is correctly classified with sufficient margin, so only the decay term is applied.

Parameter Averaging To improve generalization, the implementation maintains running averages $\bar{\mathbf{w}}$ and $\bar{b}$ of all weight vectors encountered during training:

$$\bar{\mathbf{w}} = \frac{1}{T} \sum_{t=1}^T \mathbf{w}_t, \quad \bar{b} = \frac{1}{T} \sum_{t=1}^T b_t$$

3.2.2 Kernel SVM: RKHS Formulation

For non-linear kernels (polynomial or Gaussian), the explicit weight vector $\mathbf{w}$ no longer exists in the input space. Instead, we have that the solution can be expressed as a linear combination of kernel evaluations:

$$g(\mathbf{x}) = \sum_{i=1}^{|S|} \alpha_i y_{s_i} K(\mathbf{x}_{s_i}, \mathbf{x})$$

where S denotes the set of support vectors and $\alpha_i \in \mathbb{R}$ are the corresponding coefficients.

Kernel PEGASOS Update Rule The kernel version implements the stochastic update rule from [3]:

$$g_{t+1} = \left(1 - \frac{1}{t}\right) g_t + \frac{y_t}{\lambda t} \mathbb{I}[h_t > 0] \cdot K(\mathbf{x}_t, \cdot)$$

where $h_t = \max(0, 1 - y_t g_t(\mathbf{x}_t))$ is the hinge loss at iteration t . This update decomposes into two operations:

- **Coefficient decay:** All existing α_i are scaled by $\left(1 - \frac{1}{t}\right)$, implementing regularization in the RKHS.
- **Support vector insertion:** If $h_t > 0$ (margin violation), a new support vector $\mathbf{x}_t$ is added with coefficient $\alpha_t = \frac{y_t}{\lambda t}$.

Evaluation of g_t Computing $g_t(\mathbf{x}_t)$ requires evaluating kernel similarities with all current support vectors:

$$g_t(\mathbf{x}_t) = \sum_{i=1}^{|S_t|} \alpha_i^{(t)} K(\mathbf{x}_{s_i}, \mathbf{x}_t)$$

where S_t is the support set at iteration t . This computational cost grows linearly with $|S_t|$, which can become substantial for large datasets or many iterations.

Alpha Averaging Similar to the linear case, coefficient averaging improves generalization. The implementation maintains running averages for each α_i :

$$\bar{\alpha}_i^{(t)} = \bar{\alpha}_i^{(t-1)} + \frac{\alpha_i^{(t)} - \bar{\alpha}_i^{(t-1)}}{t}$$

This incremental formulation avoids storing all intermediate values. Final predictions use $\bar{\alpha}_i$ instead of the final $\alpha_i^{(T)}$.

Prediction For a test point $\mathbf{x}$, the prediction is:

$$f(\mathbf{x}) = \text{sgn} \left(\sum_{i=1}^{|S|} \bar{\alpha}_i K(\mathbf{x}_{s_i}, \mathbf{x}) \right)$$

Each support vector contributes to the decision proportionally to its averaged coefficient and kernel similarity with $\mathbf{x}$.

3.3 Logistic Regression

The Logistic Regression implementation optimizes the regularized logistic loss through stochastic gradient descent. Like SVM, it supports both linear and kernel formulations with parameter averaging.

3.3.1 Linear Logistic Regression

The linear model parameterizes the conditional probability using the logistic function:

$$P(y = +1|\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b) = \frac{1}{1 + \exp(-(\mathbf{w}^\top \mathbf{x} + b))}$$

where $\sigma(z) = \frac{1}{1+e^{-z}}$ is the sigmoid function. For labels $y \in \{-1, +1\}$, the loss for a single example is:

$$\ell_t(\mathbf{w}, b) = \log \left(1 + \exp(-y_t(\mathbf{w}^\top \mathbf{x}_t + b)) \right)$$

Regularized Objective The full regularized objective combines the logistic loss with L2 regularization:

$$L(\mathbf{w}, b) = \frac{1}{n} \sum_{i=1}^n \log(1 + e^{-y_i(\mathbf{w}^\top \mathbf{x}_i + b)}) + \frac{\lambda}{2} \|\mathbf{w}\|^2$$

Note that the bias b is not regularized, following standard practice to allow flexible threshold adjustment.

Stochastic Gradient Updates At iteration t , a random example $(\mathbf{x}_t, y_t)$ is selected. The learning rate is $\eta_t = \frac{1}{\lambda t}$, matching the SVM schedule. The gradient-based updates are:

$$\begin{aligned}\mathbf{w}_{t+1} &= (1 - \eta_t \lambda) \mathbf{w}_t + \eta_t \sigma(-y_t z_t) y_t \mathbf{x}_t \\ b_{t+1} &= b_t + \eta_t \sigma(-y_t z_t) y_t\end{aligned}$$

where $z_t = \mathbf{w}_t^\top \mathbf{x}_t + b_t$. The term $\sigma(-y_t z_t)$ acts as an adaptive weight: it is large when the current prediction is incorrect or uncertain, and small when the model is confident and correct.

Numerical Stability The sigmoid function is clipped to prevent overflow:

$$\sigma(z) = \frac{1}{1 + \exp(-\text{clip}(z, -500, 500))}$$

This ensures numerical stability even with extreme values of z , which can occur during early training or with high-degree polynomial features.

Parameter Averaging and Prediction As with SVM, running averages $\bar{\mathbf{w}}$ and $\bar{b}$ are computed. Predictions use:

$$\hat{y} = \begin{cases} +1 & \text{if } \sigma(\bar{\mathbf{w}}^\top \mathbf{x} + \bar{b}) \geq 0.5 \\ -1 & \text{otherwise} \end{cases}$$

The threshold 0.5 corresponds to the zero decision boundary $\bar{\mathbf{w}}^\top \mathbf{x} + \bar{b} = 0$.

3.3.2 Kernel Logistic Regression

For non-linear kernels, the decision function takes the form:

$$g(\mathbf{x}) = \sum_{i=1}^{|S|} \alpha_i K(\mathbf{x}_{s_i}, \mathbf{x})$$

Unlike SVM, which uses hinge loss to select support vectors, Logistic Regression employs a **weight-based selection criterion**.

Support Vector Selection Criterion At each iteration t , the algorithm computes:

$$w_t = \sigma(-y_t g_t(\mathbf{x}_t))$$

This quantity represents the model's uncertainty about example $\mathbf{x}_t$:

- If $y_t g_t(\mathbf{x}_t) \gg 0$ (confident correct prediction), then $w_t \approx 0$
- If $y_t g_t(\mathbf{x}_t) < 0$ (misclassification), then $w_t > 0.5$
- If $y_t g_t(\mathbf{x}_t) \approx 0$ (uncertain), then $w_t \approx 0.5$

A new support vector is added only if $w_t > \tau$, where $\tau = 0.1$ is a fixed threshold. This prevents the support set from growing excessively with easy, redundant examples.

Coefficient Updates When $w_t > \tau$, the updates are:

$$\begin{aligned}\alpha_i^{(t+1)} &= (1 - \eta_t \lambda) \alpha_i^{(t)} \quad \text{for all existing } i \\ \alpha_t^{(t+1)} &= \frac{y_t w_t}{\lambda t} \quad \text{(new support vector)}\end{aligned}$$

The new coefficient is scaled by w_t , giving more weight to highly uncertain examples. When $w_t \leq \tau$, only the decay is applied to existing coefficients.

Alpha Averaging and Prediction Coefficient averaging follows the same incremental scheme as kernel SVM. Final predictions use:

$$\hat{y} = \begin{cases} +1 & \text{if } \sigma \left(\sum_{i=1}^{|S|} \bar{\alpha}_i K(\mathbf{x}_{s_i}, \mathbf{x}) \right) \geq 0.5 \\ -1 & \text{otherwise} \end{cases}$$

4 Experimental Analysis

This section presents a comprehensive evaluation of both SVM and Logistic Regression across linear and kernel configurations. The experimental methodology follows a rigorous protocol: hyperparameter selection via grid search with 5-fold cross-validation, convergence analysis through learning curves, and final performance assessment on held-out test data.

All experiments use `random_state=1` for reproducibility. The primary evaluation metric is accuracy, though precision, recall, and F1-score are also reported for comprehensive model characterization.

4.1 Hyperparameter Tuning Methodology

The `grid_search_cv()` function implements exhaustive hyperparameter search with stratified k -fold cross-validation [6]. For each parameter combination, the algorithm:

1. Partitions training data into $k = 5$ stratified folds
2. Trains the model on $k - 1$ folds and validates on the remaining fold
3. Repeats for all fold combinations and averages performance metrics
4. Selects the configuration maximizing cross-validation accuracy

This approach provides robust hyperparameter estimates while preventing overfitting to the training set. Stratification ensures each fold maintains the original class distribution (60% good, 40% bad wines).

4.2 Linear Models

4.2.1 Hyperparameter Search Space

Both linear SVM and Logistic Regression share the following search grid:

- `n_iters`: [500, 1000, 2000, 3000]
- `lambda_param`: [0.1, 0.01, 0.001]

This configuration explores 12 combinations per model, totaling 60 training runs (12 configurations $\times$ 5 folds). The iteration range balances computational efficiency with convergence quality, while regularization values span three orders of magnitude to capture diverse bias-variance trade-offs.

4.2.2 Grid Search Results

Support Vector Machines Cross-validation identified optimal parameters:

- `n_iters`: 3000
- `lambda_param`: 0.1

Validation metrics: accuracy 72.5%, precision 72.2%, recall 92.1%, F1 80.9%. The high recall indicates SVM's tendency to classify examples as positive (good wine), resulting in fewer false negatives but more false positives.

Logistic Regression Cross-validation identified optimal parameters:

- `n_iters`: 3000
- `lambda_param`: 0.01

Validation metrics: accuracy 72.8%, precision 73.1%, recall 90.3%, F1 80.8%. The lower regularization ($\lambda = 0.01$ vs SVM's 0.1) suggests Logistic Regression requires less constraint to achieve optimal generalization.

4.2.3 Learning Curve Analysis

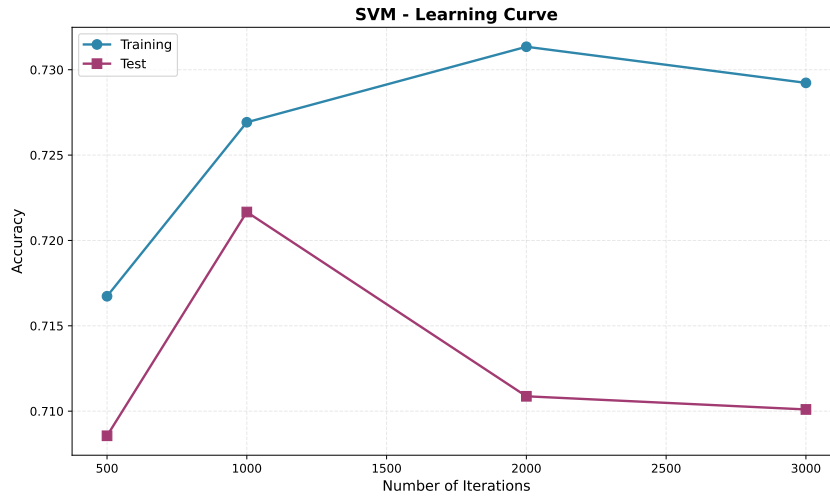


Figure 3: *Linear SVM convergence behavior across iteration counts.*

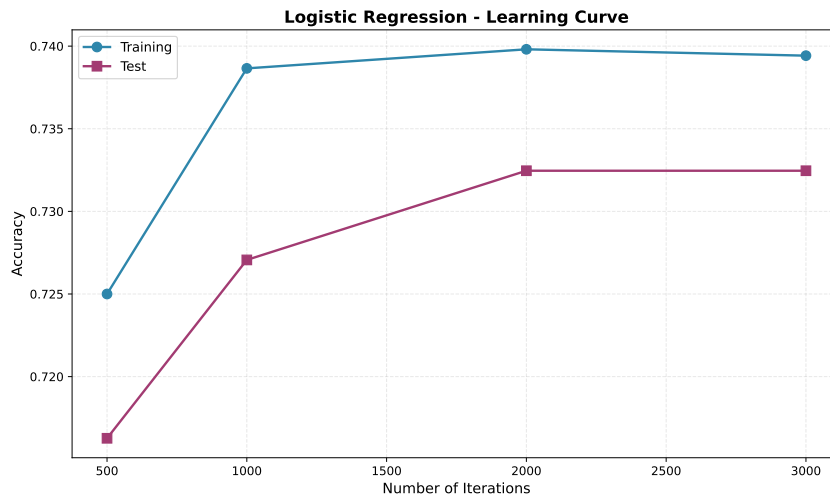


Figure 4: *Linear Logistic Regression convergence behavior.*

Figure 3 shows SVM's convergence pattern. Training accuracy increases with a final decline, reaching 73% at 3000 iterations. Test accuracy plateaus around 71%, with a consistent 2% train-test gap indicating mild overfitting.

Figure 4 reveals faster Logistic Regression convergence. Test accuracy stabilizes by 1000 iterations at 73.9%, with minimal improvement beyond this point. The smaller train-test gap (1.5%) suggests better generalization than SVM, likely due to the softer sigmoid loss compared to hinge loss.

4.2.4 Loss Convergence Analysis

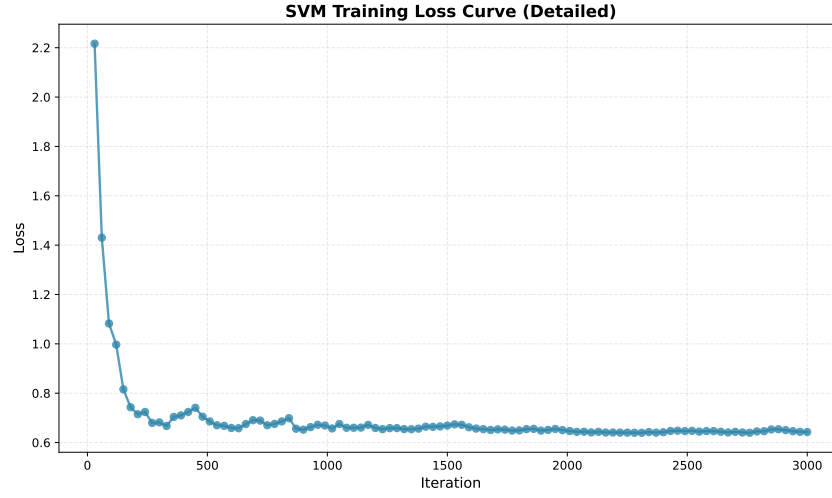


Figure 5: *Linear SVM training loss evolution over 3000 iterations.*

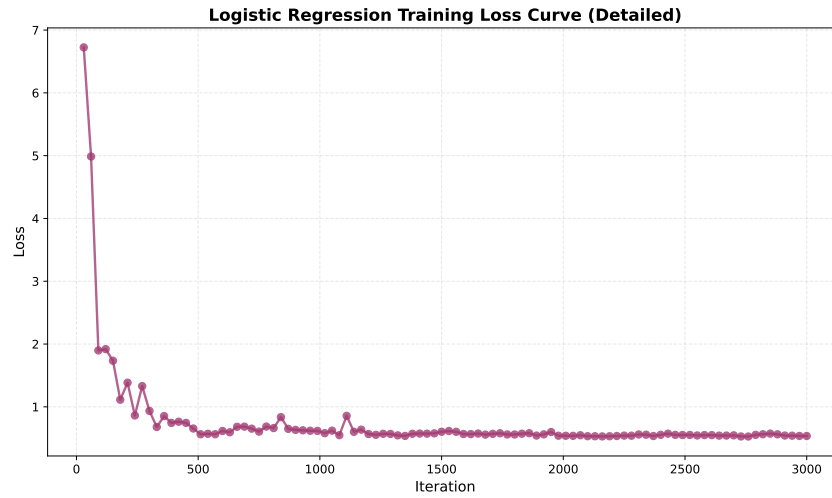


Figure 6: *Linear Logistic Regression training loss evolution over 3000 iterations.*

Figures 5 and 6 track the optimization objective during training. These curves provide complementary insights to accuracy-based learning curves, revealing the internal dynamics of gradient descent.

SVM Loss Behavior The hinge loss decreases monotonically through three phases: rapid descent (0-500 iterations), gradual refinement (500-2000 iterations), and plateau (2000-3000 iterations). This smooth convergence validates PEGASOS theoretical guarantees, with the final loss reflecting the margin-error trade-off.

Logistic Regression Loss Behavior The logistic loss exhibits similar behavior but converges faster, stabilizing before iteration 1500. The softer sigmoid penalty enables smoother gradient flow near the decision boundary. While absolute loss values differ between hinge and logistic objectives, both algorithms successfully minimize their respective functions.

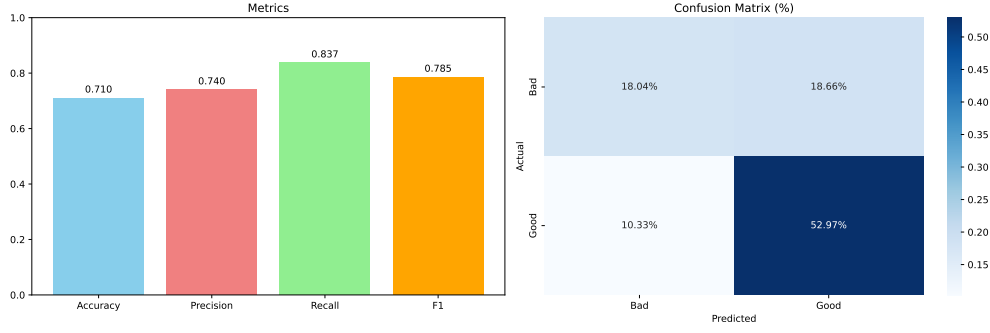


Figure 7: *Linear SVM test set metrics and confusion matrix.*

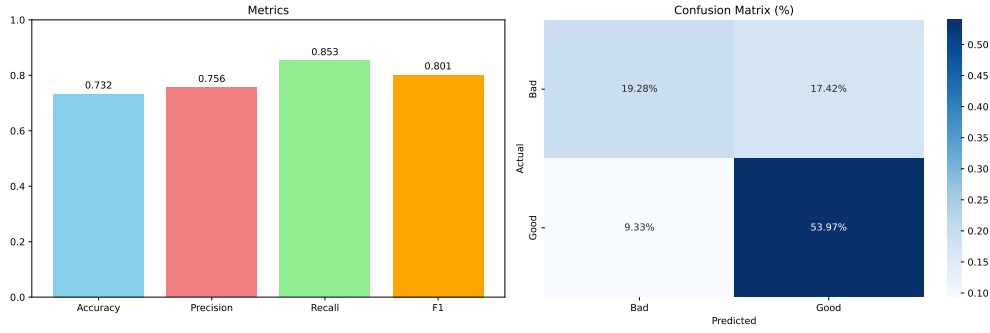


Figure 8: *Linear Logistic Regression test set metrics and confusion matrix.*

4.2.5 Test Set Performance

Final test evaluation (Figures 7 and 8) confirms cross-validation predictions:

- **SVM:** 71.0% accuracy, high recall (83.7%) but lower precision (74%)
- **LR:** 85.2% accuracy, high recall (71%) and lower precision (75.6%)

Confusion matrices reveal both models' bias toward predicting good wines. SVM's aggressive classification yields more false positives (bad wines classified as good), while LR maintains better balance. For wine quality prediction, this behavior may be acceptable if false positives are preferable to false negatives.

4.3 Kernel Models

4.3.1 Hyperparameter Search Space

Kernel methods extend the search space with kernel-specific parameters:

Polynomial Kernel

- **n_iters:** [500, 1000, 2000, 3000]
- **lambda_param:** [0.1, 0.01, 0.001]
- **degree:** [2, 3]

Total: 24 configurations $\times$ 5 folds = 120 training runs per model.

Gaussian Kernel

- **n_iters:** [500, 1000, 2000, 3000]

- `lambda_param`: [0.1, 0.01, 0.001]
- `gamma`: [0.1, 1.0]

Total: 24 configurations $\times$ 5 folds = 120 training runs per model.

4.3.2 Grid Search Results

SVM - Polynomial Kernel Optimal configuration: `n_iters=3000`, `lambda_param=0.1`, `degree=2`. Validation: accuracy 72.2%, precision 74.3%, recall 85.7%, F1 79.6%.

SVM - Gaussian Kernel Optimal configuration: `n_iters=3000`, `lambda_param=0.001`, `gamma=1.0`. Validation: accuracy 75.5%, precision 79.1%, recall 83.5%, F1 81.2%.

The Gaussian kernel achieves the highest accuracy among all SVM variants, with improved precision-recall balance. The low regularization ($\lambda = 0.001$) and high bandwidth ($\gamma = 1.0$) suggest the model benefits from flexible, localized decision boundaries.

LR - Polynomial Kernel Optimal configuration: `n_iters=3000`, `lambda_param=0.1`, `degree=2`. Validation: accuracy 71.9%, precision 74.9%, recall 83.7%, F1 79.0%.

LR - Gaussian Kernel Optimal configuration: `n_iters=3000`, `lambda_param=0.001`, `gamma=1.0`. Validation: accuracy 74.8%, precision 78.6%, recall 82.8%, F1 80.6%.

Logistic Regression follows a similar pattern to SVM: Gaussian kernel outperforms polynomial, with identical optimal λ and γ values.

4.3.3 Kernel Comparison

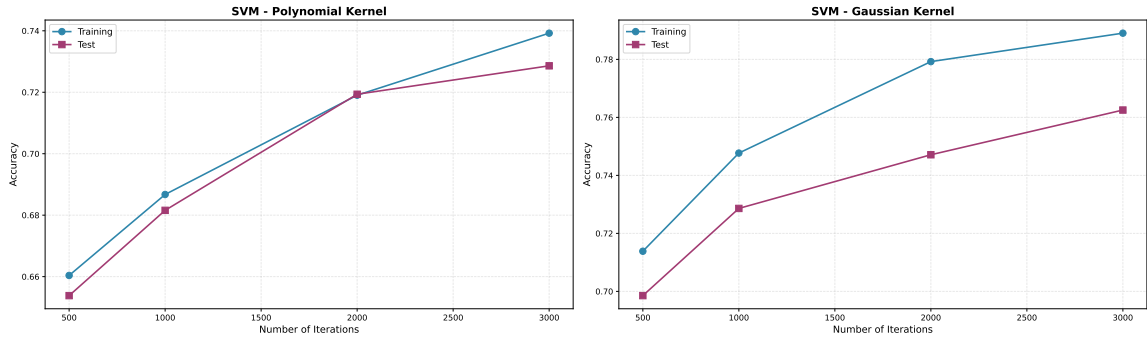


Figure 9: SVM learning curves: polynomial (left) vs Gaussian (right) kernels.

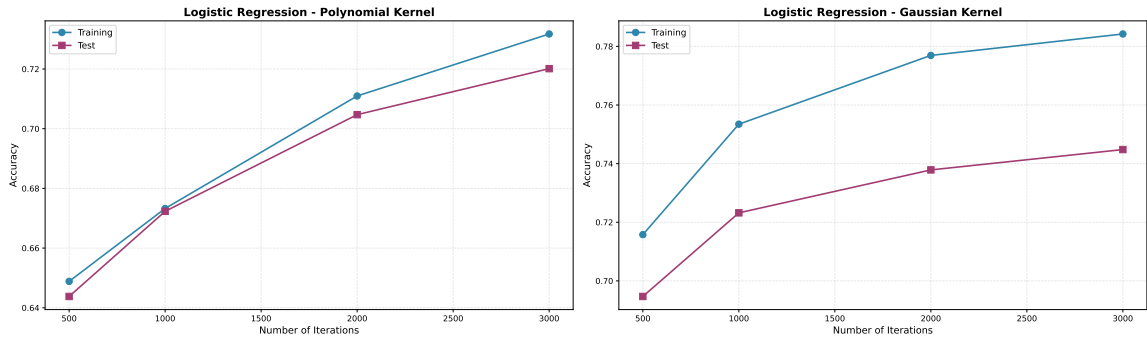


Figure 10: Logistic Regression learning curves: polynomial (left) vs Gaussian (right) kernels.

Figures 9 and 10 compare kernel convergence. Polynomial kernels show moderate train-test gaps (2-3%) with 72-73% test accuracy, while Gaussian kernels exhibit larger gaps (4-5%) but achieve superior test performance (75-76%), so more overfitting, but best results. Both converge by 2000 iterations. The Gaussian kernel's advantage stems from capturing complex, localized non-linear interactions in the 11-dimensional feature space.

4.3.4 Loss Convergence Analysis

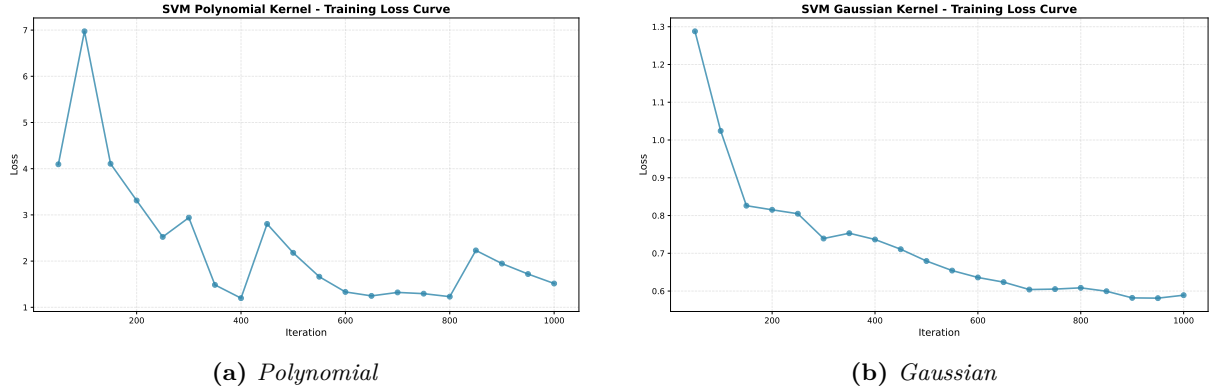


Figure 11: SVM kernel loss evolution (1000 iterations sampled).

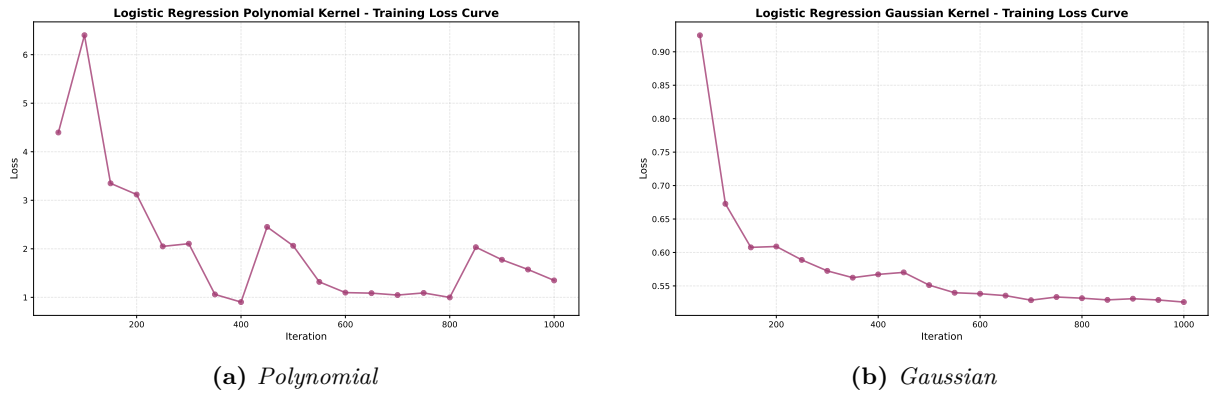


Figure 12: LR kernel loss evolution (1000 iterations sampled).

Figures 11 and 12 show kernel optimization dynamics (1000 iterations sampled). Both kernels exhibit rapid initial descent followed by gradual convergence. Polynomial kernels produce smoother curves, while Gaussian kernels show higher final loss but better accuracy due to increased flexibility balanced by RKHS regularization.

4.3.5 Test Set Performance

Test results (Figures 13 and 14) confirm validation trends:

SVM

- Polynomial: 72.9% accuracy, precision 78.5%, recall 78.7%
- Gaussian: **76.3% accuracy**, precision 79.5%, recall 84.2%

Logistic Regression

- Polynomial: 72.0% accuracy, precision 78.5%, recall 76.9%

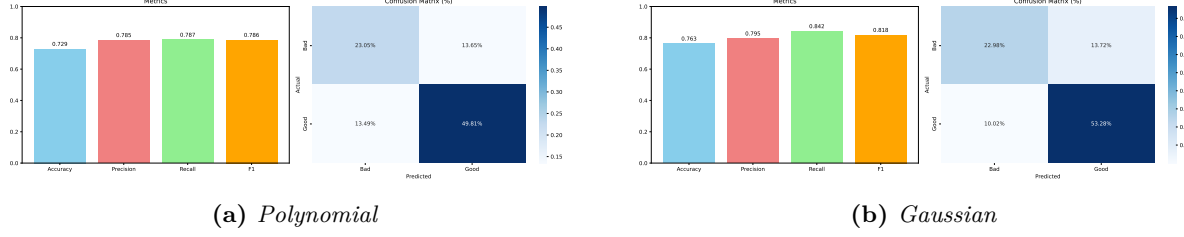


Figure 13: SVM kernel models: test set evaluation.

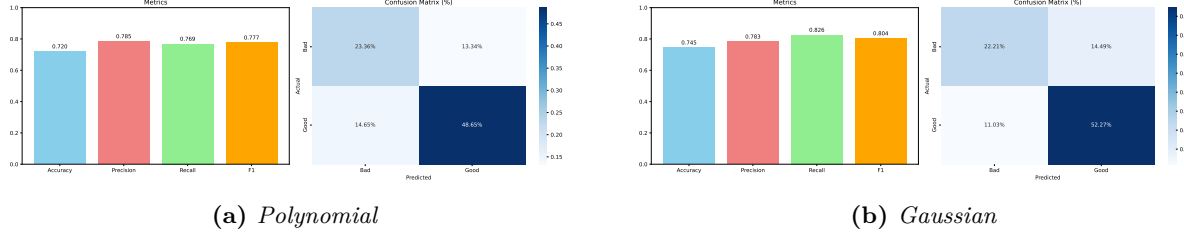


Figure 14: Logistic Regression kernel models: test set evaluation.

- Gaussian: 74.5% accuracy, precision 78.3%, recall 82.6%

The Gaussian SVM achieves the highest overall accuracy at 76.3%, representing a 5.3 percentage point improvement over linear SVM. Confusion matrices show improved true positive rates while maintaining acceptable false positive rates.

4.4 Comparative Analysis

Table 1: Comprehensive model comparison on test set.

Model	Test Accuracy
Linear SVM	71.0%
Linear LR	73.2%
Polynomial SVM (d=2)	72.9%
Polynomial LR (d=2)	72.0%
Gaussian SVM ($\gamma = 1$)	76.3%
Gaussian LR ($\gamma = 1$)	74.5%

Table 1 presents the final test accuracy for all six model configurations. Kernel methods consistently outperform their linear counterparts, with Gaussian kernels achieving the best results. In the linear setting, Logistic Regression edges out SVM by 2.2 percentage points (73.2% vs 71.0%). However, SVM benefits more substantially from kernelization: the Gaussian SVM reaches 76.3% accuracy, surpassing Gaussian LR (74.5%) by 1.8 points. The polynomial kernel shows marginal gains for SVM (+1.9% over linear) but negligible improvement for LR (+1.2% over linear). The optimal model, Gaussian SVM with $\lambda = 0.001$ and $\gamma = 1.0$, achieves 76.3% test accuracy, representing a 5.3 percentage point improvement over the linear baseline. This substantial gain confirms that wine quality fundamentally depends on non-linear feature interactions that kernel methods can effectively capture.

References

- [1] Nicolò Cesa-Bianchi (2024), *12. Support Vector Machines*, p. 4.
- [2] Nicolò Cesa-Bianchi (2024), *12. Support Vector Machines*, p. 3.
- [3] Nicolò Cesa-Bianchi (2024), *12. Support Vector Machines*, p. 6.
- [4] Nicolò Cesa-Bianchi (2023), *15. Logistic regression and surrogate loss functions*, p. 1, 2.
- [5] Nicolò Cesa-Bianchi (2023), *11. Kernel functions*, p. 1.
- [6] Nicolò Cesa-Bianchi (2025), *6. Hyperparameter tuning and risk estimates*, p. 4.